

Arkitektur principper i praksis

M9.03 - Monitorering af applikationer

Formål

I dette opgavesæt skal I anvende `Helm` til at deployment af en `Loki+Grafana+Promtail` monitorerings løsning i det `minikube`-cluster som i oprettede i **opgave M7.03**.

I starter med at installere `Helm`-værktøjet på jeres udviklings-maskine og til sidst suppleres med et simpelt katalog service som skal logge til monitoreringsløsningen.

Forberedelse

Inden du går i gang med opgaverne herunder, sørg for at **docker-desktop er startet**.

Sørg også for at dit `minikube`-cluster er oppe at køre med:

```
$ minikube start          # Start din minikube klynge
$ kubectl get pods -A     # Tjek at kubectl virker
```

kubectl

Mangler du en reference guide til `kubectl` kan du finde en her:

<https://kubernetes.io/docs/reference/kubectl/quick-reference/>

Installation af Helm

Start med at installere `Helm` på din udviklingsmaskine. Du kan følge instrukserne herunder eller [Helm's Installations guide](#).

Fra en terminal kør én af følgende:

```
# Installér Helm på Windows
$ winget install Helm.Helm

# Installér Helm på MacOS
$ brew install helm
```

Hent Grafana Helm repo-information

Start med at opdatere din lokale `helm`-database, så den kan finde mulige templates (`charts`) for Grafana-værktøjerne. Dette gøres ligeledes fra en **Bash-terminal**.

```
# Tilføj adresse for Grafana Helm repository
$ helm repo add grafana https://grafana.github.io/helm-charts

# Opdater Helm repository information
$ helm repo update

# Opdater lokale Helm repositories
$ helm search repo loki
```

Den sidste kommando lister adskillige muligheder. Vi vælger `grafana/loki-stack` i denne opgave.

Hent parameter-liste til Grafana + Loki Chart

I skal nu hente parameter-listen som I skal bruge til at deploye jeres `helm` chart . Start med at oprette en ny folder til at indeholde `yaml` -filerne til løsningen. Kald den f.eks. `k8s-test-env` . Gå ind i folderen og køр følgende kommando:

```
$ helm show values grafana/loki-stack > loki-custom-values.yaml
$ ls -la
```

Du har nu en parameter-liste med default værdier som er nødvendige til at oprette en Grafana+Loki -løsning. Den indeholder også parametre til andre devops-værktøjer, så vi skal have filen tilpasset lidt, inden vi anvender den.

Tilpasning af chart

Åben filen `loki-custom-values.yaml` med dit favorit redigeringsværktøj og find linjen med `grafana: -servicen` (ca. linje 36).

- Start med at aktivere den, ved at sætte `enabled` til `true`.
- I skal også sætte service-typen. Lav en ny *array*, på niveau med `image:` som hedder `service:`. Den skal have en `chart-værdi` som hedder `type:` som indeholder værdien `NodePort`.

Din `grafana:` gruppe bør nu se således ud:

```
grafana:
  enabled: true
  sidecar:
    datasources:
      label: ""
      labelValue: ""
      enabled: true
      maxLines: 1000
  image:
    tag: 10.3.3
  service:
    type: NodePort
```

Deploy af chart

I skal nu udføre et deployment til klyngen med jeres `chart`. Vi opretter også et afskærmet namespace til løsningen, kaldet `monitoring`. Fra en terminal kør følgende `helm`-kommando:

```
$ helm upgrade --install --values loki-custom-values.yaml loki grafana/loki-stack \
  -n monitoring --create-namespace
```

Kode-analyse

Brug `helm upgrade --help` til at analysere parametrene på den `helm upgrade`-kommando du lige har udført.

Kig i Dashboard'et

Find de nye **K8s-objekter** med minikube's dashboard. Husk at vælge det rigtige `namespace` i drop-down boksen øverst i billedet.

Find dine Grafana værdier

Du skal nu finde dine legitimationsdata til Grafana. Brug en terminal til at køre de følgende kommandoer for at finde dit brugernavn og password i K8s hemmeligheds-database:

1. Tjek hvilke nye `services` er dukket op med kommandoen
`kubectl get services -n monitoring`. Tjek at du har en service som hedder `loki-grafana` på listen.
2. Find dit Grafana **brugernavn** med:

```
$ kubectl get secret loki-grafana -n monitoring \
  -o jsonpath="{.data.admin-user}" | base64 --decode
```

3. Find dit Grafana **password** med:

```
$ kubectl get secret loki-grafana -n monitoring \
  -o jsonpath="{.data.admin-password}" | base64 --decode
```

4. Find **URL'en** for din `Grafana` -service og forbind den til en browser med:

```
$ minikube service loki-grafana -n monitoring --url=false
```

✓ Test din Grafana-løsning

Det er nu tid til at tjekke at din `Grafana` -applikation fungerer. Hvis punkt 4 i foregående trin fungerede korrekt har `minikube` åbnet en browser op med Grafana's loginside. Brug `brugernavn` og `password` fra forrige trin til at logge på med.

Denne udgave af `Grafana` kommer prækonfigureret med en `data source` til den lokale `Loki` -service. Så vi kan gå direkte til at udforske dataene uden at lave en ny `data source` :

1. Fra hamburger-menuen i venstre side vælg `connections` .
2. I `Connections` -menuen vælg `Data Sources`
3. Vælg `Explore` -knappen ude til højre i billedet, så du kan komme videre til dit søge-vindue

4. Under `Label filters`, vælg en label, f.eks. `app = grafana`
5. Tryk `Run Query` -knappen øverst få at udføre din log-anmodning

Vent med at bygge et dashboard til du har deploy'et en microservice (se herunder).

</> Eksempel Catalog microservice

Deploy følgende `catalog-service` til din `minikube` -klynge og lav et `Grafana` -dashboard som viser log informationen fra applikationen.

Du skal brug følgende fil kaldet `catalog-svc.yaml` :

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: catalog-service
spec:
  selector:
    matchLabels:
      run: catalog-service
  replicas: 1
  template:
    metadata:
      labels:
        run: catalog-service
    spec:
      containers:
        - name: catalog-service
          image: hnrkjnsn/catalog-service:1.0.1
          env:
            - name: ImagePath
              value: "/srv"
          ports:
            - containerPort: 8080
              protocol: TCP
```

Kør følgende kommandoer for at lægge dem ind i klyngen:

```
$ kubectl apply -f catalog-svc.yaml
$ kubectl expose deployment catalog-service --type=NodePort --name=catalog-service
$ minikube service catalog-service --url # få URL til catalog-service
$ curl http://<URL>/catalog/version # Hent version fra catalog-service
```

Kildekoden til `catalog-service` finder du på dette github link:

<https://github.com/Handel-og-Vandel/CatalogService.git>

? Rollback og feature disable

Undersøg hvordan du kan løse følgende problemstillinger:

- Design en test til at undersøge om `CatalogService` har fejl på `ImageController` metoderne.
- Hvordan laver du et **rollback** af det foregående deployment?
- Hvordan kan du **disable** en feature som `uploadImage` i `ImageController` ? Undersøg [🔗 .NET Feature Management](#) for en mulig løsning?

